阳光学习能量站 详细设计说明书

版本：v1.13 ｜ 编号：LLD-SUN-2026-08 ｜ 上游：SRS-SUN-2026-08、HLD-SUN-2026-08 ｜ 实现：阳光学习能量站.html（含 FR-10~FR-24；FR-15 多孩子、FR-16 周期任务、FR-13 导入加固、FR-17 庆祝动效、FR-18 里程碑/完美周、FR-19 内容编辑器拖拽排序、FR-20 亮/暗主题切换、FR-21 可选 WebDAV/JSON 云同步、FR-22 数据层健壮性（阳光对账 + ledger 滚动裁剪）、FR-23 课本差量化存储（curriculumByGrade 由全量深拷贝改为偏离差量）、FR-24 机密加密（pin PBKDF2 单向哈希 + sync.pass AES-GCM 信封加密，加载透明迁移、旧数据兼容；v1.11 加固：解密失败返回 null、verifyPin 空串语义收紧、迁移范围扩至所有非空明文 pin 且启动段 await 后再渲染、KMS 模块级缓存、savePin 防重入；v1.12 非功能优化：P0 属性转义+导入白名单、P1 勾选局部重渲染、P2 分发映射表/CURRICULUM 课本常量集中/家长中心汇总浮层；v1.13 功能增强（O12 今日推荐 daily 优先稳定排序）+ O17 复核确认已在 FR-16 阶段闭环））

1. 引言

1.1 编写目的

在《需求规格说明书（SRS）》与《概要设计说明书（HLD）》基础上，对系统各模块的内部结构、函数级行为、数据结构字段、关键算法流程、出错分支与界面元素给出可执行级别的详细描述，作为编码实现（已落地为单文件 ■■■■■■■.html）与测试验证的直接依据。

1.2 范围

- 覆盖 HLD 定义的 6 个模块（Storage / State / View / Controller / Seed / Util）的详细设计。
- 字段级数据结构、localStorage 物理结构、种子数据结构。
- 关键算法精确流程与边界条件。
- 本文与已实现代码保持一一对应；函数名、常量名以代码为准。

1.3 术语

同 SRS/HLD。补充：

- 闭包/IIFE：本文实现未使用模块打包，全部以全局函数 + 全局变量 `state`、`currentSubject` 组织。
- 事件委托：在 `document` 上统一监听 `click` / `change`，通过 `data-act` / `data-*` 属性路由。

1.4 参考资料

- SRS-SUN-2026-08、HLD-SUN-2026-08（同目录）。
- 实现文件 ■■■■■■■.html（<script> 段）。
- 原始文章（今日头条，约 2026-08）。

---

2. 常量与全局变量

名称	类型	说明
`STORAGE_KEY`	string	固定 `"sunshine_station_v1"`，localStorage 唯一键。
`LEVELS`	array	等级阈值表（默认源）：`[{min:0,name:"Lv.1 ■■"},…{min:600,"Lv.5 ■■"}]`；`buildDefaultState` 经 `defaultLevels()` 生成带 `id` 的 `state.levels`（可由 FR-14 编辑，全局共享）。
`MILESTONES`	array	连续打卡里程碑奖励表（FR-18）：`[{days:7,bonus:50},{days:30,bonus:100},{days:100,bonus:200}]`；`checkIn` 在 `streakDays` 达到时一次性发放。

名称	类型	说明
PERFECT_WEEK_BONUS	int	完美周奖励阳光值（FR-18）：60；连续 7 个完美日发放一次后 perfectStreak 归零。
LEDGER_KEEP	int	ledger 滚动窗口上限（FR-22）：1000；超过则 pruneLedger 将最早部分汇总为单条 aggregate 锚点，保持 Σamount === sunshine 守恒，存储恒定有界。
makeDelta/applyDelta	function	FR-23 课本差量化：makeDelta(subjects, grade) 以 defaultSubjects(grade) 为基准仅提取偏离学科小树（含 subjects/added/removed 三部分），未编辑年级返回 null；applyDelta(delta, grade) 由默认种子 + 差量还原等价课本，旧版整树数组兼容 clone。
PBKDF2_ITER	int	FR-24 PIN 哈希迭代次数：100000；配合随机 16B salt 经 PBKDF2-SHA256 派生 256bit 哈希，抵御离线暴力。
KMS_STORAGE_KEY	string	FR-24 每安装随机 AES-GCM 密钥的 localStorage 键："sunshine_station_kms"（独立于主存储键，使导出/复制主 JSON 不含密钥，提升静态保护）。
HAS_CRYPTO	bool	FR-24 运行期 Web Crypto 可用性守卫：!!crypto?.subtle && !!crypto.getRandomValues；为假时 hashPin/encryptSecret 退化为 {v:0, alg:"none"} 且不崩溃。
hashPin/verifyPin/encryptSecret/decryptSecret/loadKms/migrateSecrets/hasPin	function	FR-24 机密加密：PIN 单向 PBKDF2 哈希与校验、sync.pass AES-GCM 信封加密与还原、KMS 密钥加载、加载期明文→密文透明迁移、hasPin 判定审批码是否已设（兼容对象/明文）。
PALETTE	array	学科配色调色板（10 个 hex），供 addSubject 循环取色与编辑面板颜色选择。
state	object	内存唯一可信状态 AppState（见 §4）；含全局 rewards/levels 与 children[]；state.children[i] 各持独立 subjects/curriculumByGrade/ledger。
currentSubject	string	当前选中学科 id，默认 "yuwen"；仅 UI 态，不入持久化。
pendingRewardId	string|null	审批码弹窗中暂存的待兑换奖励 id。
theme	string	全局主题（"light"/"dark"，默认 "light"），FR-20；ensureIds 将非法值归一化为 "light"。
dragCtx	object|null	拖拽进行中的上下文 {kind,subjId,unitId,id}（FR-19）；onDragEnd 清空。
suppressDirty	bool	同步过程中（如 syncPush/applyRemote）置 true，使 save() 不误将本地标记 dirty（FR-21）。
autoPushTimer	int|null	自动同步 debounce 定时器句柄（FR-21）；scheduleAutoPush 防抖约 1.5s。
state.sync	object	可选云同步配置（FR-21，默认 defaultSync()）：enabled/url/user/pass/rememberPass/auto/dirty/lastSync；仅当 enabled=true 才发起网络请求，离线优先不变。
cur() / activeChildId	fn / string	cur() 返回 state.children.find(c=>c.id===activeChildId)
SUBJECT_COLORS	object	8 学科配色表（hex）。
GRADE_LABELS	object	{ "1":"■■■", …, "6":"■■■" }。

名称	类型	说明
GRADE_YUWEN / GRADE_SHUXUE / GRADE_YINGYU	object	按年级（key "1"~"6"）的语文/数学/英语单元数组（每单元含 tasks 字符串数组）。
COMMON_UNITS	object	科学/道法/体育/音美/综合 的通用单元（不区分年级）。

---

3. 模块详细设计

3.1 Util 模块（工具函数，纯函数）

函数	签名	输入	输出	逻辑
levelOf	levelOf(s:int)->string	阳光值	等级名	遍历 LEVELS，取 s>=min 中 min 最大者；默认 Lv.1 ■■。
levelIndexOf	levelIndexOf(s:int)->int	阳光值	等级序号	遍历 LEVELS，取 s>=min 的最高序号；用于 addSunshine 升级检测。
todayStr	todayStr()->string	无	"YYYY-MM-DD"	取本地 Date，月份/日期 padStart(2,"0")。
diffDays	diffDays(a,b)->int	两日期串	天数差 b-a	任一为空返回 99（视为"非连续"）；以 T00:00:00 解析后按 86400000ms 取整。
escapeHtml	escapeHtml(s)->string	文本	转义文本	转义 & < > " '，防 XSS（任务标题/姓名渲染用）。

3.2 Seed 模块（默认数据）

• defaultSubjects(grade='5')→subjects[]：按年级构造样例（原 buildSeed 内联逻辑抽出）。语文/数学/英语取 GRADE_*、其余取 COMMON_UNITS（均 clone() 后给 unit.id=subjId+"-u"+ui 与 task.id=subjId+"-u"+ui+"-t"+ti，done:false，recur:"once"）。
• getSubjectsForGrade(g)→subjects[]：若 cur().curriculumByGrade[g] 为差量对象则返回 applyDelta(delta，g)（=defaultSubjects(g)+差量还原），为旧版整树数组则 clone() 返回（向后兼容），均不存在则返回 defaultSubjects(g)。供 switchGrade 与 load 复用（FR-23）。
• buildChild(name,grade,pin)→child：生成单个孩子档案：{ id，name，grade，pin，sunshine:0，streakDays:0，lastCheckinDate:null，lastDailyReset:null，awardedMilestones:[]，dayLog:{}，perfectStreak:0，lastPerfectDate:null，subjects:defaultSubjects(grade)，curriculumByGrade:{}，ledger:[] }（后四个字段为 FR-18 里程碑/完美周状态）。
• buildDefaultState()→AppState：构造 meta+全局 rewards(defaultRewards())+levels(defaultLevels())+sync:defaultSync()+children:[buildChild("■■","5","")]+activeChildId=children[0].id（FR-15/FR-21）。
• defaultSync()→sync：返回默认同步配置 { enabled:false，url:""，user:""，pass:""，rememberPass:false，auto:false，dirty:false，lastSync:null }（FR-21）。
• 学科样例内容（见 HLD §4.1，与实现一致）：内置小学 1–6 年级；五年级为语文统编五上 8 单元、数学 5 单元、英语 4 单元、科学 3 单元；其余年级提供同结构单元骨架，科学/道法/体育/音美/综合为通用模板。

3.3 Storage 模块（持久化）

函数	签名	行为
`load()`	`load()`	读取 `STORAGE_KEY`；成功则 `JSON.parse` → `ensureIds()` → `reconcileBalance()`（FR-22 阳光对账护栏：账本非空且 `sunshine` 与账本派生余额不一致时以账本为准修复，空账本安全不动、避免清零真实阳光）→ `pruneLedger(cur())`（FR-22 ledger 滚动裁剪）→ `reconcileStreak()` → `resetRecurringTasks()`；解析失败或为空则 `state=buildDefaultState()` 并 `save()`、重置。加载成功后另在顶层调用 `migrateSecrets()`（FR-24 机密迁移：明文 `pin`/`sync.pass` 透明升级为哈希/密文，已是对象则跳过，无 Web Crypto 不动作）。
`save()`	`save()`	`localStorage.setItem(key, JSON.stringify(state))`；`try/catch` 失败则 `toast("[redacted]…")`，仅内存可用。若 `state.sync.enabled && !suppressDirty`：置 `state.sync.dirty=true`（标记自上次同步有本地改动）；若同时 `state.sync.auto` 则触发 `scheduleAutoPush()` debounce 自动上传（仅当 `rememberPass` 时后台有凭据可认证，否则静默跳过）。
`exportFile()`	`exportFile()`	以 `Blob` 生成 JSON 文件，文件名 `[redacted]-<today>.json`，触发 `<a download>` 下载；`URL.revokeObjectURL` 释放。
`validateImport(d)`	`validateImport(d:object)`	校验 `d.children` 为非空数组（拒绝旧版单 `profile`，提示迁移）；每名孩子的 `subjects` 为数组、`id/name` 为 string、`units` 为数组、单元 `title` 为 string、`tasks` 为数组、任务 `title` 为 string；`d.rewards`/`d.levels` 为数组；并校验规模上限（孩子≤20、学科≤20、单元≤50、任务≤200、总任务≤500）；任一不满足 `throw new Error`。供 `importFile` 在 `confirm` 前调用（FR-13 加固）。
`importFile(file)`	`importFile(file:File)`	`FileReader.readAsText`；`onload` 内 `JSON.parse` → `validateImport(data)`（结构非法/超限抛错）→ 若 `confirm("[redacted]…")` 取消则 return（不覆盖）；用 `defaultRewards()`/`defaultLevels()` 补全、`ensureIds()` 对每个孩子的任务补 `recur`（默认 `"once"`）并对重复 `task.id` 追加 `-N` 后缀去重为唯一；成功则 `state=data; ensureIds(); reconcileStreak(); resetRecurringTasks(); activeChildId=[redacted]; save(); renderAll()`；异常则 `toast("[redacted]")`，不覆盖（FR-13 加固）。
`printReport()`	`printReport()`	遍历 `cur().subjects` 生成「学科→单元→任务勾选框」的 HTML 清单（含当前孩子姓名、年级、阳光值、等级、连续打卡天数、日期），写入 `#printArea`（`innerHTML`），调用 `window.print()`；`@media print` 仅渲染 `#printArea`，隐藏顶栏/导航/商店/按钮（FR-12）。

3.4 State 模块（业务状态与变更）

函数	签名	行为 / 边界
`ensureIds()`	`ensureIds()`	遍历 `state.children`，对每个孩子补全其 `subjects` 缺失的 `unit.id`/`task.id`、`recur`(默认 `"once"`)、`lastDailyReset`(默认 null)、`awardedMilestones`(默认 `[]`)、`dayLog`(默认 `{}`)、`perfectStreak`(默认 `0`)、`lastPerfectDate`(默认 null)；补 `rewards`/`levels`(默认带 id)/`curriculumByGrade:{}`/`ledger` 默认；补 `theme`（默认 `"light"`，非法值归一化为 `"light"`，FR-20）；保证旧数据兼容（旧版 `profile.subjects` 结构不再接受）。
`switchGrade(g)`	`switchGrade(g)`	若 `g===cur().grade` 直接 return；`confirm` 覆盖确认 → `const delta = makeDelta(cur().subjects, cur().grade)`（FR-23 差量化：仅存偏离默认种子的学科小树，未编辑返回 `null` 则不写）→ 若 `delta` 则 `cur().curriculumByGrade[grade]=delta`，否则 `delete` → `cur().subjects=getSubjectsForGrade(g)`（`applyDelta` 默认种子 + 差量 / 旧版数组克隆 / 默认样例）→ `cur().grade=g; currentSubject=cur().subjects[0].id; ensureIds(); save(); renderAll()`（FR-14 升级 FR-10：课本编辑按年级保留，按孩子隔离；FR-23 存储精简）。

函数	签名	行为 / 边界
requestRedeem(id)	requestRedeem(id)	若 cur().pin=="" 引导家长设置（openPinSettings）并返回；否则 pendingRewardId=id 并弹出审批码输入框（pinEntry）。
submitPin()	submitPin()	比对输入与 cur().pin；不符 → toast("■■■■■") 返回；相符 → 关闭弹窗、redeem(pendingRewardId)、celebrate({type:"redeem", text:"■■■■■■"})（FR-17 兑换成功动效）。
savePin()	savePin()	校验新码为 4 – 6 位数字且两次一致；若已设码须校验当前码；写入 cur().pin; save()。
openPinSettings()	openPinSettings()	清空输入框并显示审批码设置面板（pinSettings）。
reconcileStreak()	reconcileStreak()	遍历各孩子：若其 lastCheckinDate 存在且 diffDays(last, today)>=2 → streakDays=0 且 awardedMilestones=[]（断签归零并清空已发里程碑，可重新累计再获；各孩子独立，FR-18）。
addSunshine(amount, type,note)	addSunshine(a,t,n)	统一阳光变更入口（FR-17/FR-18）：before=levelIndexOf(cur().sunshine)；cur().sunshine+=amount；pushLedger(t, amount, n)；after=levelIndexOf(cur().sunshine)；若 after>before → celebrate({type:"levelup", text:"■■■ "+levelOf(cur().sunshine)+"■"})。任务/打卡/兑换/里程碑/完美周均经此，保证 ledger.balance 与阳光值一致。
updatePerfectWeek()	updatePerfectWeek()	完美周检测（FR-18）：统计当日 recur:"daily" 任务完成度 done/total，写 cur().dayLog[today]={perfect}；若当天完美且 lastPerfectDate!=today：与上一完美日相邻(diffDays==1)则 perfectStreak+1 否则置 1，更新 lastPerfectDate；perfectStreak>=7 → addSunshine(PERFECT_WEEK_BONUS,"perfectweek","■■■■■") + celebrate({type:"perfectweek",...}) 并 perfectStreak=0（可循环）；最后剪枝 dayLog（仅留近 60 天）。
celebrate(opts)	celebrate(opts)	庆祝动效（FR-17）：纯 DOM——若 typeof document==="undefined"
toggleTask(taskId)	toggleTask(id)	在 cur().subjects 全量查找任务；未找到直接 return；翻转 done；amount = done?+5:-5；addSunshine(amount, "task", done?"■■■"+title:"■■■"+title)；updatePerfectWeek()；save(); renderAll()。
checkIn()	checkIn()	t=todayStr()；若 cur().lastCheckinDate===t → toast("■■■■■■■■") 并 return（同日防重）；d=diffDays(cur().lastCheckinDate,t)；d>=2 → cur().streakDays=0; cur().awardedMilestones=[]；cur().streakDays+=1；cur().lastCheckinDate=t；addSunshine(20,"checkin","■■■■")；遍历 MILESTONES：若 streakDays>=m.days && awardedMilestones 不含 m.days → 记入并 addSunshine(m.bonus,"milestone",m.label+"■■") + celebrate({type:"milestone",...})；updatePerfectWeek()；save(); renderAll()；toast 连续天数。
redeem(rewardId)	redeem(id)	取奖励；cur().sunshine<cost → toast("■■■■■■■ X■■") return；否则 addSunshine(-cost,"redeem","■■■"+title)；save(); renderAll(); renderStore()。（实际扣减由 submitPin 在审批通过后调用。）
setName(n)	setName(n)	去除首尾空格，空则 "■■■"；写入 cur().name；save(); renderTopbar()。
pushLedger(type,amount,note)	pushLedger(...)	向 cur().ledger 追加 {ts:ISOString, type, amount, balance:cur().sunshine, note}。balance 恒等于变更后该孩子 sunshine，作为对账锚点；末尾调用 pruneLedger(cur())（FR-22 滚动裁剪），保持存储有界。
reconcileBalance()	阳光对账护栏（FR-22）	遍历 state.children：若孩子 ledger 非空，计算账本派生余额 refBalance = ■■.balance(回退 Σamount)；若 cur().sunshine !== refBalance 则修正 sunshine=refBalance 并 pushLedger("repair", 0, ...)（仅记录修复事件，不改真实累计，Σamount 守恒）。ledger 为空直接返回——无依据，保留现有 sunshine，不臆造或清零。纯存储层函数，不动任何业务阳光逻辑。

函数	签名	行为 / 边界
pruneLedger(child)	ledger 滚动裁剪（FR-22）	若 child.ledger.length > LEDGER_KEEP(1000)：keep=■1000■、remove=■■；sum=Σ(remove.amount)；anchorBalance = keep[0].balance - keep[0].amount；生成 anchor={ts:remove[0].ts, type:"aggregate", amount:sum, balance:anchorBalance, note:"■■■■(N■)"}，最终 ledger=[anchor, ...keep]。裁剪后 Σamount 与 sunshine 严格守恒，运行余额链连续（anchorBalance+keep[0].amount === keep[0].balance）。运行期每次 pushLedger 末尾与 load() 中均调用，存储恒定有界。
getRecommendation(n)	getRecommendation(n=3)	按"学科顺序→单元顺序"收集 done===false 任务（带 subj/subjName/title），slice(0,n)；不足 n 则全取。
computeProgress(subjId)	computeProgress(id)	该学科 done/total*100 取整；无任务返回 0。
addSubject() / renameSubject(id,name) / setSubjectColor(id,color) / removeSubject(id)	学科增/改名/改色/删	removeSubject 在 subjects.length<=1 时拦截并 toast；删除经 confirm；删当前学科则 currentSubject 指首项。均 save()+renderAll()(+renderEditor())。
addUnit(subjId) / renameUnit(subjId,unitId,title) / removeUnit(subjId,unitId)	单元增/改名/删	单元最少保留 1 项（units.length<=1 拦截）；删除经 confirm。
addTask(subjId,unitId) / renameTask(subjId,unitId,taskId,title) / removeTask(subjId,unitId,taskId) / setRecur(subjId,unitId,taskId,recur)	任务增/改名/删/设周期	新增任务 done:false, recur:"once" 并分配稳定 id（含时间戳防重）；setRecur 设 recur:"once"
addReward() / updateReward(id,field,val) / removeReward(id)	奖励增/改/删	field∈{title,cost}；cost 解析为非负整数；rewards.length<=1 拦截删除。
addLevel() / updateLevel(id,field,val) / removeLevel(id)	等级增/改/删	新增 min=max+50；按 min 升序排序；levels.length<=1 拦截删除。
cur() / switchChild(id) / addChild(name) / removeChild(id) / resetRecurringTasks()	多孩子与每日重置核心	cur() 取当前孩子档案（FR-15 访问器）；switchChild 设 activeChildId+持久化+重渲染；addChild 经 prompt 取名字生成新档案并 push；removeChild 保留≥1 否则拦截；resetRecurringTasks() 对每名孩子若 lastDailyReset!=today 则将其 recur:"daily"&&done 任务重置 done=false 并置 lastDailyReset=today（FR-16，仅清勾选不扣阳光）。
reorder(ctx,targetId,after) / moveItem(arr,from,to,after)	拖拽重排（FR-19）	reorder 按 ctx.kind∈{subject,unit,task} 在 cur().subjects / findSubject(subjId).units / findUnit(subjId,unitId).tasks 数组内 moveItem 重排；跨层级 ctx 经 findSubject/findUnit 判空忽略，数据不乱序；落定 save()+renderAll()+renderEditor()。moveItem(arr,from,to,after) 用 splice 先删后插（因删除修正目标索引偏移，after 表示插入到目标之后），为纯函数便于单测。
toggleTheme() / applyTheme()	主题切换（FR-20）	toggleTheme 翻转 state.theme（light⇄dark），save()+renderTopbar()+applyTheme()+toast；applyTheme 将 state.theme 写入 document.documentElement[data-theme]（typeof document==="undefined"
defaultSync()	同步默认配置（FR-21）	返回 { enabled:false, url:"", user:"", pass:"", rememberPass:false, auto:false, dirty:false, lastSync:null }。

函数	签名	行为 / 边界
`ensureIds()` 同步归一化（FR-21）	`ensureIds()`	在已有字段补全后：若 `state.sync` 非对象则置 `defaultSync()`；否则将 `enabled/rememberPass/auto/dirty` 归一化为布尔、`url/user/pass` 归一化为 string、`lastSync` 非 string 时置 null。
`basicAuthHeader(user,pass)`	Basic Auth 头（FR-21）	优先 `btoa(user+":"+pass)`（浏览器原生），回退 `Buffer.from(...,"binary").toString("base64")`（Node 测试环境）；返回 `"Basic "+tok`。
`buildSyncPayload()`	同步上传载荷（FR-21）	深拷贝 `state` → `meta.syncedAt=now` → 剔除 `sync.pass`（密码不上传）→ 返回载荷对象。
`syncPush(pass)`	上传到云端（FR-21）	若 `typeof fetch==="undefined"` 抛「离线模式」错误（安全降级）；`url` 为空抛「未配置同步地址」；以 `PUT` + Basic Auth 上传 `JSON.stringify(buildSyncPayload())`；非 `ok` 抛「上传失败（HTTP x）」；成功后 `state.sync.lastSync=syncedAt`、`dirty=false` 并持久化（不经 `save()`，避免误标 dirty）。
`syncPull(pass)`	从云端下载（FR-21）	`GET` + Basic Auth；`404` 抛「云端还没有文件，请先上传」；非 `ok` 抛「下载失败（HTTP x）」；`await res.json()`；`validateImport` 结构校验（非法抛「云端文件结构不完整」）；若 `state.sync.dirty && remoteAt>lastSync`（两端均有新改动）则 `confirm` 二选一，取消抛 `CONFLICT_KEEP_LOCAL`；否则 `applyRemote(data, remoteAt)`。
`applyRemote(data,remoteAt)`	覆盖本地（FR-21）	补全 `data.rewards/levels/children` 字段 → `state=data; ensureIds(); reconcileStreak(); resetRecurringTasks()` → 保留本机客户端同步配置（enabled/url/user/pass/rememberPass/auto）仅更新 `lastSync=remoteAt`、`dirty=false`（经 `suppressDirty` 期间 `save()` 避免误标）→ `renderAll(); applyTheme()`。
`scheduleAutoPush()`	自动同步（FR-21）	若已有定时器则返回；否则 `setTimeout(≈1.5s)` 后：若 `enabled && auto` 且 `rememberPass`（有凭据）则 `await syncPush(pass)`，异常静默忽略；未 `rememberPass` 时静默跳过（需手动）。

3.5 View 模块（渲染）

全部读 `state` 生成 DOM（innerHTML），不持有状态。

函数	行为
`renderTopbar()`	渲染 logo、孩子切换下拉 `<select id="childSelect">`（选项=各孩子名，change→`switchChild`）、`<input class="name-edit">`（值=`cur().name`，change→setName）、连续打卡　　、阳光☀　、等级（`levelOf`）。
`renderNav()`	渲染 8 学科按钮（当前 `currentSubject` 加 `.active`，圆点取 `color`）；底部"打卡+20"按钮（`data-act="checkin"`）。
`renderContent()`	先渲染"　　今日推荐"面板（`getRecommendation(3)`，每项点击→`select`该学科；全完成时给鼓励文案）；再渲染当前学科：标题 + 进度条（`<i style="width:pct%;background:color">`）+ 各单元卡片（任务 `<label>` + checkbox（done 加 `.done` 删除线）+ `+5■■` + 周期选择 `<select class="recur-sel" data-act="set-recur">`（once/daily，FR-16））。
`renderStore()`	渲染奖励列表，每项"兑换"按钮；`sunshine<cost` 时 `disabled` 且文案"阳光不足"。
`renderAll()`	首次注入页面骨架（topbar/layout/工具栏含 导出/导入/重置）；随后调用 4 个 render。
`escapeHtml()`	见 Util。
`toast(msg)`	顶部居中轻提示，`setTimeout` 1.8s 消失。
`printReport()`	见 §3.3 Storage；生成 A4 勾选式清单并调用 `window.print()`（FR-12）。
`openEditor()` / `renderEditor()`	「　　内容管理」入口：`openEditor()` 关闭审批码面板、打开 `#contentEditor` 浮层；`renderEditor()` 顶部含「○、孩子管理」（孩子下拉 + 新增/删除按钮，`data-act="add-child"/"del-child"`，保留≥1），再生成树形编辑 HTML（学科→单元→任务、奖励列表、等级列表），每个任务行含周期选择 `data-act="taskRecur"`；各输入带 `data-edit` 字段、`data-act="add-*/del-*"` 按钮；改名经 `change` 即时保存，增删经 `click` 调 State 编辑函数，孩子增删经 `addChild/removeChild` 后重渲染（FR-14/FR-15）。

函数	行为
openSyncPanel() / renderSyncPanel() / saveSyncConfig()	云同步面板（FR-21）：openSyncPanel() 关闭审批码面板、打开 #syncPanel 浮层并 renderSyncPanel() 回填配置（启用/地址/账号/密码(仅 rememberPass 时)/记住密码/自动同步/上次同步时间）；saveSyncConfig() 读取输入写入 state.sync、enabled 无地址则回落关闭、auto 未 rememberPass 则回落关闭，并 save()+renderSyncPanel()+toast。
doSyncTest() / doSyncPush() / doSyncPull()	云同步动作（FR-21）：doSyncTest() 以 PROPFIND（Depth:0）请求，按 401/404/207 等状态码反馈凭据/可达性；doSyncPush() 取密码（rememberPass 用存值，否则用输入框）→ await syncPush → renderSyncPanel()+toast（失败按 CONFLICT/其他分类提示）；doSyncPull() 取密码 → await syncPull → 成功 renderSyncPanel()，冲突取消提示「已保留本地」、其他失败提示原因。

3.6 Controller 模块（交互控制）

• 全局 click 委托：e.target.closest("[data-act]") 取动作；select→切 currentSubject+重渲染；checkin→checkIn()；open-store→显示商店；close-store→隐藏；open-pinset→打开审批码设置（close-store 先关）；close-pinset/save-pin→保存；close-pinentry→关闭并清 pendingRewardId；submit-pin→submitPin()；redeem→requestRedeem(id)（须审批）；export→exportFile()；import→doImport()；reset→doReset()；open-editor→openEditor()、close-editor→关编辑浮层、add-subject/del-subject/add-unit/del-unit/add-task/del-task/add-reward/del-reward/add-level/del-level→对应 State 编辑函数（FR-14）；add-child/del-child→addChild/removeChild（FR-15）；toggle-theme→toggleTheme()（FR-20）；open-sync→openSyncPanel()、close-sync→关同步浮层、save-sync→saveSyncConfig()、sync-test→doSyncTest()、sync-push→doSyncPush()、sync-pull→doSyncPull()（FR-21）。

• 年级下拉 change 委托：gradeSelect 的 change → switchGrade(value)。

• 孩子切换下拉 change 委托：childSelect 的 change → switchChild(value)（FR-15）。

• 全局 change 委托：[data-act="toggle"] 的 checkbox → toggleTask(id)（避免 click 重复触发，由 change 统一处理）；[data-edit] 字段（subjectName/subjectColor/unitTitle/taskTitle/rewardTitle/rewardCost/levelName/levelMin）→ 按字段路由到 rename*/update* 即时保存（FR-14）。

• doImport()：动态创建 <input type=file accept=".json,application/json">，onchange→importFile(files[0])。

• doReset()：confirm("■■■■…") 确认后 state=buildDefaultState(); currentSubject="yuwen"; save(); renderAll()（重置同时清空为默认单孩子，审批码清空）。

• 拖拽排序事件委托（FR-19）：document 上挂 dragstart/dragover/drop/dragend 监听器，分别调用 onDragStart/onDragOver/onDrop/onDragEnd；onDragStart 记录 dragCtx={kind,subjId,unitId,id}，onDragOver/onDrop 仅在落点与起点同层级（同 subjId 或同 subjId+unitId）时 preventDefault 允许放置并据指针上/下半区决定 after，落定调用 reorder(ctx,targetId,after) 重排并 save()+重渲染；仅浏览器环境生效，非浏览器（测试桩）安全降级、不报错。

• 启动序列：load(); renderAll(); applyTheme();（FR-20 在首屏即落位 state.theme）。

---

4. 数据结构详细设计

4.1 AppState（内存 + localStorage 同一份）

```
AppState = {
  meta:    { version:1, schema:"sunshine_station" },          // int, string■■■■■
```

```
  theme:   "light",                                          // ■■■■■light/dark■FR-20■
  rewards: [ { id:string, title:string, cost:int>0 } ],       // ■■■■■■■■ FR-14 ■■
  levels:  [ { id:string, min:int>=0, name:string } ],        // ■■■■■■■■■■■■■■■ FR-14 ■■
  children: [                                                 // ■■■■■■■FR-15■
    {
      id:string, name:string,           // ■■ "■■"■■■■■
      grade:string,                      // "1".."6"■■■■■■
      pin:string,                        // ■■■■■■■4-6■■■■■"" ■■■■■■■
      sunshine:int>=0,                   // ■■■■■■■■■
      streakDays:int>=0,                 // ■■■■■■■■■■
      lastCheckinDate:string|null,       // "YYYY-MM-DD"
      lastDailyReset:string|null,        // "YYYY-MM-DD"■■■■■■■■■FR-16■
      subjects: [ {                                           // ■■■■ 8 ■
        id:string, name:string, color:string(hex),
        units:[ { title:string,
                  tasks:[ { id:string, title:string, done:bool, recur:"once"|"daily" } ] } ]
      } ],
      curriculumByGrade: { [grade:string]: { subjects:{[id]:subjectTree}, added:subjectTree[], removed:str
      awardedMilestones: [ int ],                             // ■■■■■■■■■(7/30/100)■■■■■■■■FR-18■
      dayLog: { [date:"YYYY-MM-DD"]: { perfect:bool } },      // ■■■■■■■■■■■■■FR-18■■■■■■ 60 ■
      perfectStreak:int, lastPerfectDate:string|null,         // ■■■■■■■ / ■■■■■■■FR-18■
      ledger:   [ { ts:string(ISO), type:"task"|"checkin"|"redeem"|"milestone"|"perfectweek"|"aggregate"|"
                    amount:int, balance:int, note:string } ]
    }
  ],
  activeChildId:string,                                       // ■■■■■■■ id
  sync: {                                                     // FR-21 ■■■■■■■■■■■■■■■■■■■■■■
    enabled:bool, url:string, user:string, pass:string,
    rememberPass:bool, auto:bool, dirty:bool,
    lastSync:"YYYY-MM-DDTHH:mm:ss.sssZ"|null
  }
}
```

约束与不变式

• 每名孩子的 `sunshine === ledger[ledger.length-1].balance`（对账锚点；渲染与统计以 `cur().sunshine` 为准，异常时以 ledger 末条修复，FR-22 `reconcileBalance` 在加载期自动修复漂移、空账本安全不动）。所有阳光变更统一经 `addSunshine`，保证 `balance` 与 `sunshine` 一致。

• `ledger` 为滚动窗口：明细默认保留最近 `LEDGER_KEEP=1000` 条，更早部分由 FR-22 `pruneLedger` 汇总为单条 `type:"aggregate"` 锚点（数组最前，其 `amount`=被汇总之和、`balance`=`keep[0].balance - keep[0].amount` 以承接运行余额链）；裁剪前后 `Σamount === sunshine` 严格守恒，存储恒定有界，与 FR-18 `dayLog` 仅留近 60 天的滚动裁剪风格一致。

• `streakDays` 仅由该孩子 `checkIn`/`reconcileStreak` 维护，非独立漂移；断签（`d>=2`）时与 `awardedMilestones` 一并归零，可重新累计再获里程碑（FR-18）。

• `task.id` 在同一孩子内唯一且稳定（重建/导入均经 `ensureIds`；跨孩子允许同名 id，互不干扰）。

• 每日重置仅清 `recur:"daily"` 任务的 `done`，不修改 `sunshine`/不写 ledger（FR-16）。

• 里程碑奖励（`type:"milestone"`）与完美周奖励（`type:"perfectweek"`）同样经 `addSunshine` 记账，与任务/打卡/兑换统一对账。

4.2 物理结构

• 位置：浏览器 `localStorage`，键 `sunshine_station_v1`，值为上述结构 `__check` 校验过的 JSON 字符串（含 2 空格缩进导出）。

• 无服务端、无索引库；导出文件即该 JSON 副本。

4.3 种子数据（SEED）结构

见 §3.2。年级内容存于 GRADE_YUWEN/GRADE_SHUXUE/GRADE_YINGYU（key 为 "1"~"6"），通用内容存于 COMMON_UNITS；任务 id 编码规则：{■■id}-u{■■■■■}-t{■■■■■}，便于在 toggleTask 中 O(n) 遍历定位（n 量级小，无需索引）。

---

5. 关键算法详细流程

5.1 等级换算 levelOf(s)

```
for l in LEVELS: if s >= l.min: r = l
return r.name
```

阈值可在 LEVELS 常量中调整，无需改逻辑。

5.2 连续打卡 checkIn + reconcileStreak

```
reconcileStreak():
  if lastCheckinDate != null and diffDays(last, today) >= 2: streakDays = 0

checkIn():
  t = todayStr()
  if lastCheckinDate == t: toast("■■■■■■"); return
  d = diffDays(lastCheckinDate, t)
  if d >= 2: streakDays = 0          // ■■■■■≥2■■■■
  streakDays += 1
  sunshine += 20
  lastCheckinDate = t
  pushLedger("checkin", +20, "■■■■")
```

说明：diffDays 为空返回 99，故首次打卡（last=null）走 d>=2 分支 → streakDays 先 0 后 +1 = 1，正确。

5.3 积分一致性 toggleTask / redeem

```
toggleTask(id):
  t = find(id); if !t: return
  t.done = !t.done
  amount = t.done ? +5 : -5
  sunshine += amount
  pushLedger("task", amount, ...)

redeem(id):
  r = find(id); if sunshine < r.cost: toast("■■"); return
  sunshine -= r.cost
  pushLedger("redeem", -r.cost, ...)
```

每条变更都写 ledger 且 balance=sunshine，保证任意时刻求和一致。

5.4 今日推荐 getRecommendation(n=3)

```
collect all tasks where done==false, in (subject order, unit order)
return slice(0, n)
```

稳定排序保证推荐结果可预期；全部完成则返回空（UI 给鼓励文案）。

5.5 进度百分比 computeProgress(subjId)

```
done=count(t.done), total=count(tasks) in subject
return total ? round(done/total*100) : 0
```

5.6 年级切换 switchGrade 与家长审批 requestRedeem/submitPin

```
switchGrade(g):
  if g == cur().grade: return
  confirm -> cur().curriculumByGrade[g] = makeDelta(cur().subjects, ■■■■)   // ■■■■■FR-23■■■■■■■■ null
  cur().subjects = getSubjectsForGrade(g)        // ■■■■■■■■■■■■
  cur().grade = g; currentSubject = cur().subjects[0].id
  ensureIds(); save(); renderAll()
requestRedeem(id):
  if cur().pin=="" : openPinSettings(); return
```

```
  pendingRewardId=id; ■■ pinEntry
submitPin():
  if ■■!=cur().pin : toast("■■■■■"); return
  redeem(pendingRewardId)
  celebrate({type:"redeem", text:"■■■■■■"})   // FR-17 ■■■■■■
```

审批码仅存本机，为"家长在场确认"机制；修改须校验当前码（首次设置除外）。

5.7 打印报告与导入校验 printReport / validateImport

- `printReport()`：拼接清单 HTML（`h1` 标题 + `prow` 汇总行 + 各 `psub` 学科 + `ptask` 单元/任务勾选框），写入 `#printArea`；`@media print` 仅显示 `#printArea`（A4，FR-12）。
- `validateImport(d)`：要求 `d.children` 为非空数组（拒绝旧版单 `profile`），逐项断言每名孩子的 `subjects`/`units`/`tasks` 字段结构与 `d.rewards`/`d.levels`(数组)，并校验规模上限（孩子≤20、学科≤20、单元≤50、任务≤200、总任务≤500）；任一不满足即抛错；`importFile` 在替换前对重复 `task.id` 追加 `-N` 后缀去重。`importFile` 捕获后提示且不覆盖（FR-13 加固）。

5.8 内容动态维护（FR-14）

- 编辑面板 `renderEditor()` 输出树形结构，每个输入带 `data-edit=<■■>` 与 `data-id/data-subj/data-unit/data-task`，每个增删按钮带 `data-act="add-*/del-*"`。
- 改名：`change` 委托命中 `[data-edit]` → 调用 `rename*/update*`（如 `renameTask(subjId,unitId,taskId,val)`、`updateReward(id,"cost",val)`）→ `save()`+`renderAll()`（不重渲染编辑器，保留焦点）。
- 增删：`click` 委托命中 `add-*/del-*` → 调用 `add*/remove*` → `save()`+`renderAll()`+`renderEditor()`；`remove*` 对非最后一项经 `confirm`，最少保留 1 项（`subjects/units/rewards/levels` 各自长度下限）。
- 课本跨年级保留：`switchGrade` 写 `cur().curriculumByGrade[■■■■]=makeDelta(...)`（FR-23 差量覆盖，未编辑则不写）；`getSubjectsForGrade` 切回时由 `applyDelta`（默认种子 + 差量）优先还原，保证各年级、各孩子编辑互不丢失，存储体积远小于整树深拷贝。

5.9 里程碑/完美周（FR-18）与庆祝动效（FR-17）

```
checkIn():
  ... streakDays += 1 ...
  addSunshine(20, "checkin", "■■■■")
  for m in MILESTONES:                       // [{7,+50},{30,+100},{100,+200}]
    if streakDays >= m.days and m.days not in cur().awardedMilestones:
      cur().awardedMilestones.push(m.days)
      addSunshine(m.bonus, "milestone", m.label+"■■")
      celebrate({type:"milestone", text:m.label+"■■■+"+m.bonus+"■■"})

updatePerfectWeek():
  total=count(recur=="daily"); done=count(recur=="daily" and done)
  cur().dayLog[today] = {perfect: total>0 and done==total}
  if dayLog[today].perfect and lastPerfectDate != today:
    if lastPerfectDate and diffDays(lastPerfectDate, today)==1: perfectStreak += 1
    else: perfectStreak = 1
    lastPerfectDate = today
    if perfectStreak >= 7:
      addSunshine(PERFECT_WEEK_BONUS, "perfectweek", "■■■■■")
      celebrate({type:"perfectweek", text:"■■■■■■+"+PERFECT_WEEK_BONUS+"■■"})
      perfectStreak = 0                      // ■■■■■■■■■■■
addSunshine(amount,type,note):
  before=levelIndexOf(cur().sunshine); cur().sunshine+=amount; pushLedger(type,amount,note)
  if levelIndexOf(cur().sunshine) > before:
    celebrate({type:"levelup", text:"■■■ "+levelOf(cur().sunshine)+"■"})
```

• 断签（d>=2）时 reconcileStreak/checkIn 将 streakDays 与 awardedMilestones 一并归零，因此断签后重新累计可再次获得已发里程碑（避免"永久只能领一次"的挫败感）。
• 中断（隔天或非完美日）使 perfectStreak 从下一个完美日重计 1，不会凭空延续。
• celebrate 在无 DOM 环境（typeof document==="undefined" || !document.body）安全返回，纯视觉、零网络。

5.10 拖拽排序（FR-19）与主题切换（FR-20）

```
onDragStart(e):
  el = e.target.closest("[data-drag]"); if !el: return
  dragCtx = {kind: el.data-drag, subjId, unitId, id}
onDragOver(e):
  block = ■ kind ■■■■ .ed-subj/.ed-unit/.ed-task
  if !block: return
  if ■■■(subjId/unitId ■■): return        // ■■■■■■■■■■
  e.preventDefault(); dropEffect="move"
onDrop(e):
  block = ■■■■■; targetId = block ■■■ id
  if targetId == dragCtx.id: clearDrag(); return
  after = (e.clientY - block.top) > block.height/2   // ■/■■■■■■■■■
  reorder(dragCtx, targetId, after); clearDrag()
reorder(ctx, targetId, after):
  arr = kind==subject ? cur().subjects
       : kind==unit   ? findSubject(ctx.subjId).units
       : findUnit(ctx.subjId, ctx.unitId).tasks
  moveItem(arr, findIndex(id==ctx.id), findIndex(id==targetId), after)
  save(); renderAll(); renderEditor()
moveItem(arr, from, to, after):
  if from==to: return
  item = arr.splice(from,1)[0]
  dest = to; if dest>from: dest-=1; if after: dest+=1
  clamp(dest); arr.splice(dest,0,item)
```

• 拖拽仅改变数组顺序，字段结构不变；reorder/moveItem 为纯函数，可在测试桩中直接以 reorder({kind,id,subjId,unitId}, targetId, after) 调用验证（T31：学科/单元/任务均可移动到首位/指定之后，跨层级调用为 no-op）。
• 主题：theme 存于 state 顶层（默认 "light"，ensureIds 将非法值归一化为 "light"）；toggleTheme 翻转并 save()+renderTopbar()+applyTheme()；applyTheme 写入 <html data-theme> 激活 CSS 变量覆盖 [data-theme="dark"]，启动与导入后均落位，刷新保留（零依赖、零网络）。

5.11 可选云同步（FR-21）

```
buildSyncPayload():
  payload = deepcopy(state)
  payload.meta.syncedAt = nowISO()
  delete payload.sync.pass          // ■■■■■
  return payload

syncPush(pass):
  if fetch undefined: throw "■■■■"
  if url empty: throw "■■■■■■■■"
  res = PUT(url, {Authorization: Basic(user,pass), body: payload})
  if !res.ok: throw "■■■■(HTTP x)"
  state.sync.lastSync = payload.meta.syncedAt
  state.sync.dirty = false
  persist()                          // ■■ save()■■■■■ dirty

syncPull(pass):
  res = GET(url, {Authorization: Basic(user,pass)})
  if res.status==404: throw "■■■■■■■■■■■■■"
  if !res.ok: throw "■■■■(HTTP x)"
  data = res.json(); validateImport(data)        // ■■■"■■■■■"
```

```
  remoteAt = data.meta.syncedAt
  remoteNew = remoteAt && (!lastSync || remoteAt>lastSync)
  if state.sync.dirty && remoteNew:
    if !confirm("■■■■■■■■■■…"): throw CONFLICT_KEEP_LOCAL
  applyRemote(data, remoteAt)

applyRemote(data, remoteAt):
  state = data; ensureIds(); reconcileStreak(); resetRecurringTasks()
  ■■■■ sync ■■(enabled/url/user/pass/rememberPass/auto)
  state.sync.lastSync = remoteAt || now; state.sync.dirty = false
  suppressDirty=true; save(); suppressDirty=false
  renderAll(); applyTheme()

scheduleAutoPush():
  if timer: return
  setTimeout(1.5s): if enabled&&auto&&rememberPass: await syncPush(pass)  // ■■■■■■
```

• 默认 `enabled=false`，离线优先不变；启用需家长在面板显式配置地址/账号/密码。

• 无 `fetch` 环境（测试桩）下 `syncPush/syncPull` 直接抛「离线模式」错误且绝不崩溃；凭证仅 `rememberPass` 时明文存本机，UI 警示「仅在你信任的设备勾选」；上传载荷剔除 `pass`。

• 冲突策略：本地 `dirty` 且云端 `syncedAt` 更新 → 弹窗二选一；非此情形直接云端覆盖本地（本地无改动则无感）。

5.12 数据层健壮性（FR-22）

```
reconcileBalance():
  for c in state.children:
    if c.ledger ■■: continue            // ■■■■■■■■■■■■■■■■■
    sum = Σ(e.amount for e in c.ledger)
    refBalance = (■■.balance ■ number) ? ■■.balance : sum
    if c.sunshine != refBalance:
      c.sunshine = refBalance             // ■■■■■■■■
      c.ledger.push({ts:now, type:"repair", amount:0, balance:c.sunshine, note:"■■■■■■■"})

pruneLedger(c):
  if c.ledger.length <= LEDGER_KEEP: return
  keep   = c.ledger.slice(-LEDGER_KEEP)
  remove = c.ledger.slice(0, len - LEDGER_KEEP)
  sum    = Σ(remove.amount)
  anchorBalance = keep[0].balance - keep[0].amount
  anchor = {ts:remove[0].ts, type:"aggregate", amount:sum, balance:anchorBalance, note:"■■■■(N■)"}
  c.ledger = [anchor, ...keep]
```

• `reconcileBalance` 仅在加载期（`load()` 内 `ensureIds` 之后）执行；`repair` 条 `amount:0` 故 `Σamount` 守恒、不改变真实累计，仅记录修复事件。

• `pruneLedger` 运行期每次 `pushLedger` 末尾与 `load()` 中均调用，保证存储恒定有界且 `Σamount === sunshine` 不变量始终成立；重复调用幂等。

5.13 课本差量化存储（FR-23）

```
makeDelta(subjects, grade):
  base = defaultSubjects(grade)
  baseById = {s.id: s for s in base}
  delta = { subjects:{}, added:[], removed:[] }
  seen = {}
  for s in subjects:
    seen[s.id] = true
    if s.id in baseById:
      if not sameSubject(s, baseById[s.id]): delta.subjects[s.id] = stripSubject(s)   // ■■■■■■■■■■■■
    else:
      delta.added.push(stripSubject(s))                                              // ■■■■■■■■■■
  for s in base:
    if s.id not in seen: delta.removed.push(s.id)                                    // ■■■■■■■■■
  if delta ■■: return null                                                          // ■■■■■■■■■
  return delta
applyDelta(delta, grade):
```

```
  result = clone(defaultSubjects(grade))
  result = result.filter(s => s.id not in delta.removed)
  for id, sub in delta.subjects:                                                    // ■■■■■■
    idx = result.findIndex(s => s.id === id); if idx>=0: result[idx] = normalizeSubject(sub, id, ...)
  for sub in delta.added:                                                           // ■■■■■■
    if sub.id not in result: result.push(normalizeSubject(sub, sub.id, null))
  return result
```

• `sameSubject(a,b)` 比较 `id/name/color/units/tasks`（含 `done/recur`），故勾选 `done` 视为偏离、随差量保留；`stripSubject/normalizeSubject` 保证差量结构与默认种子同构（`done` 缺省 `false`、`recur` 缺省 `once`、`color` 兜底）。

• 旧版 `curriculumByGrade[g]` 为整树数组，`getSubjectsForGrade` 检测数组直接 `clone` 返回（原行为不变）；新写入走差量对象，旧用户下次 `switchGrade` 时旧数组被 `makeDelta` 自然覆盖迁移。

• 非浏览器/无 DOM 环境（测试桩）下 `makeDelta/applyDelta` 仅做纯数据运算，不触发渲染，零网络。

---

6. 界面（UI）详细设计

区域	元素	交互
顶部栏	logo、孩子切换器 、姓名输入框、年级选择器 、 家长按钮、连续打卡 、阳光☀等级	孩子切换 `change→switchChild`（FR-15）；姓名 `change→setName`；年级 `change→switchGrade`； 家长→`openPinSettings`
左侧导航	8 学科按钮（圆点配色）+ 打卡按钮	点击→`select/checkin`；当前学科 `.active`
右侧内容 · 推荐	3 张推荐卡（任务名+学科）	点击→跳转该学科
右侧内容 · 学科	标题 + 进度条（按学科色）+ 单元卡片	勾选/取消→`toggleTask`
右下浮层 fab	兑换商店	打开浮层（overlay）
浮层 · 商店	奖励项 + 兑换按钮（不足置灰）+ 家长设置入口	`requestRedeem`（须审批）
审批码设置面板	当前码 / 新码 / 确认新码 输入	`savePin`（校验长度与当前码）
审批码输入弹窗	兑换前输入 4 – 6 位审批码	`submitPin`（校验后 `redeem`）
底部工具栏	导出 / 导入 / 打印/A4 / 重置	分别触发对应动作；打印生成 A4 勾选清单；重置带 `confirm`
家长设置面板	审批码（当前/新/确认）+「内容管理」按钮 +「☁ 云同步设置」按钮	审批码 `savePin`；内容管理→`openEditor`；云同步→`openSyncPanel`
云同步设置浮层	启用开关 / WebDAV 地址 / 账号 / 密码 / 记住密码 / 自动同步 / 状态行 / 测试连接 · 上传 · 下载按钮	`save-sync→saveSyncConfig`；`sync-test→doSyncTest`（PROPFIND 验证）；`sync-push→doSyncPush`（PUT 含 syncedAt、剔除密码）；`sync-pull→doSyncPull`（GET→validateImport→冲突二选一→applyRemote）；默认关闭，离线优先不变（FR-21）
内容管理浮层	「◯、孩子管理」(孩子下拉 + 新增/删除，保留≥1) + 学科→单元→任务树形编辑、奖励列表编辑、等级列表编辑	孩子增删 `click→addChild/removeChild`（FR-15）；改名 `change` 即时保存；增删 `click` 调编辑函数；删除 `confirm` 保护、最少保留 1 项
全局	toast 轻提示	操作反馈

响应式：≤720px 时导航转为横向滚动，布局纵向堆叠。

5.14 机密字段加密（FR-24）

• PIN 单向哈希 `hashPin(pin)`：无 Web Crypto 时返回 `{v:0, alg:"none", hash:pin}`（降级）；否则 `crypto.getRandomValues(16B salt)` → `importKey("raw", pin, "PBKDF2", false, ["deriveBits"])` → `deriveBits({name:"PBKDF2", salt, iterations:100000, hash:"SHA-256"}, key, 256)` → 返回 `{v:1, alg:"pbkdf2-sha256", salt:bytesToB64(salt), hash:bytesToB64(bits)}`。`verifyPin(input, stored)`：未设置（null/undefined）返回 `true`（无审批码）；空串要求 `input===""`（与原 `v!==c.pin` 一致，避免空码被任意输入绕过）；明文串走旧版直接比对；`{v:0}` 走明文兜底（`input===stored.hash`）；`{v:1}` 用 `stored.salt` 重算并 `ctEq` 定长比较。

• 同步密码信封加密 `encryptSecret(plain, kms)`：`loadKms()` 读/生成并持久化 32B 随机 KMS 密钥（独立键 `KMS_STORAGE_KEY`）；`crypto.getRandomValues(12B iv)` → `importKey("raw", kms, {name:"AES-GCM"}, false, ["encrypt"])` → `encrypt({name:"AES-GCM", iv}, key, encode(plain))` → `{v:1, alg:"aes-gcm", iv:bytesToB64(iv), ct:bytesToB64(ct)}`；无 crypto/kms 时 `{v:0}`（降级）。`decryptSecret(stored, kms)`：明文串原样返回（旧版兼容）；`{v:0}` 返回 `ct`；`{v:1}` `importKey(...["decrypt"])` → `decrypt` → `TextDecoder` 得明文，失败返回 `null`（区别于「未配置」，供 `doSyncPush`/`doSyncPull` 提示「本地密钥已丢失，请重新输入密码」）。`loadKms()` 首次读/生成并持久化 32B 随机 KMS 密钥（独立键 `KMS_STORAGE_KEY`）后模块级缓存 `_kmsKey` 复用，避免每次加解密重复解析 localStorage。

• 透明迁移 `migrateSecrets()`：启动段先 `await` 完成后再 `renderAll()`（先迁移后渲染，消除「迁移未完成时用户操作把明文写回」竞态窗口）；`HAS_CRYPTO` 为假直接返回（保留旧明文、由兼容路径兜底）；否则对仍为任意非空字符串的 `pin` 字符串原地 `hashPin`（不再限于 4 – 6 位数字），对非空 `sync.pass` 字符串原地 `encryptSecret`，有变更才写回 `localStorage`。旧数据无需脚本即平滑升级；导入/未迁移的明文数据经兼容路径照常工作。

• 安全降级边界：`HAS_CRYPTO=false` 时所有加密边界退化为 `{v:0}` 结构且不抛错，功能可用（仅失去静态加密）。单文件离线应用无主密码，信封加密防的是「随手 inspect / 导出文件外传」导致的凭证泄露，不防同设备已取得完整 localStorage 的攻击者——此边界已在需求与 UI 文案中明示。

• v1.11 加固项：`savePin` 增加 `pinSaveBusy` 防重入锁（try/finally 复位），避免快速双击并发改写 `c.pin`；`verifyPin` 空串语义收紧（仅 null/undefined 放行）；`decryptSecret` 失败返回 `null` 并区分未配置；`migrateSecrets` 启动段 `await` 后再渲染、迁移范围扩至所有非空明文 pin；KMS 密钥模块级缓存 `_kmsKey` 复用。功能规格不变。

• v1.12 优化项（非功能，功能规格不变）：

  - P0 安全转义（O1）：新增 `safeColor()`（`/^#?[0-9a-fA-F]{3,8}$/.test` 放行、否则回落默认色），所有 `style="background:..."` 注入点改用 `safeColor(s.color)`；`subject/task/unit/reward/level/child` 的 `id` 与 `name` 在拼接 HTML 属性值（含 `data-*`、`value=`、`option value=`）前统一 `escapeHtml`；`validateImport` 在遍历校验中增 `child.id`/`s.id`/`t.id` 的 `^[A-Za-z0-9_-]+$` 白名单与 `s.color` 的 `safeColor` 一致性校验，非法即 `throw`（深度防御）。 - P1 性能（O3b）：`toggleTask` 末尾由 `save();renderAll();` 改为 `save();renderContent();renderTopbar();`（若 `contentEditor` 处于 `show` 再补 `renderEditor();`），避免每次勾选重建 nav/store/整树。 - P2 结构（O7）：原 `document.addEventListener("click", …)` 内 40+ 分支 `if/else if` 重构为模块级常量 `DISPATCH = { select:()=>{…}, … }`，分发体仅 `const h =`

DISPATCH[act]; if(h) h();（其中 toggle 不登记，由 change 事件单独处理），纯行为保持、可维护性提升。 - P2 内容解耦（O8）：原散落的 GRADE_YUWEN/GRADE_SHUXUE/GRADE_YINGYU/COMMON_UNITS/SUBJECT_COLORS 五个顶层常量，包进单一 CURRICULUM = { GRADE_YUWEN, GRADE_SHUXUE, GRADE_YINGYU, COMMON_UNITS, SUBJECT_COLORS } 命名空间（上方加「内容配置区」边界注释，标明「替换为其他地区/年级课本只需改此块」）；defaultSubjects(grade) 改经 CURRICULUM.GRADE_YUWEN[grade] 等访问。为保护 Node 回归桩直接读 JS 常量，未抽成 <script type="application/json"> 独立块。 - P2 UX（O11）：顶栏 pin-btn 文案「家长」→「 ┆ ┆ 家长中心」、action 改 open-hub；新增 openParentHub()/closeParentHub() 与 #parentHub 浮层（CSS 复用 .overlay/.panel），内含审批码/内容/同步/主题/导出/导入/打印/重置 8 个入口按钮（各自 data-act 落到既有 open-pinset/open-editor/open-sync/toggle-theme/export/import/print/reset）；openPinSettings/openEditor/openSyncPanel 开头均 closeParentHub()，进子面板自动收起 hub。 - 收窄：O4（多孩子按需物化）经复核不成立已取消（buildChild 仅 switchGrade 时加载当前年级树，其余年级由 curriculumByGrade 差量还原，并非每人常驻全树）；O3a（save 防抖）暂缓（save() 在 Node 桩内同步写 localStorage，防抖会破坏"立即读"断言，且无新引入的过频写盘）。

- v1.13 功能增强（O12）+ 复核（O17）：
  - O12 今日推荐增强：getRecommendation 收集未完成任务时附加 daily:(t.recur==="daily") 标记，随后 list.sort((a,b)=>(b.daily?1:0)-(a.daily?1:0)) 稳定排序（daily 优先，组内保持原序）；因无截止日期字段，"临期"落地为"每日必做优先"。 - O17 复核：经全文件审计，resetRecurringTasks()（遍历 children 比对 lastDailyReset，非当日则把 recur==="daily" && done 任务重置）已在 load()（852）、doImport()（1585）、applyRemote()（1675）调用，lastDailyReset 在 buildDefaultState 初始化并兜底，FR-16 阶段即已闭环，原评审误列未做已更正。

7. 出错处理详细设计

场景	触发	处理
本地存储不可用/超额	save() 抛异常	toast 提示，内存仍可运行
导入文件非法（JSON/结构）	importFile 解析或 validateImport 校验失败	toast("■■■■■■■■■■■■■■■■■■")；保持当前数据
导入覆盖确认被取消	importFile 中 confirm("■■■■■...") 返回 false	直接 return，保持当前数据，不覆盖
兑换余额不足	redeem 前置判断	拦截、toast("■■ X■■")、不写 ledger
同日重复打卡	checkIn 判定 last===today	toast("■■■■■■")、不重复计
审批码错误	submitPin 比对不符	toast("■■■■■■")、不扣减不写流水
任务 id 缺失（旧数据）	ensureIds	按规则补全，保证可定位
删除最后一项（学科/单元/奖励/等级）	remove* 前置判断 length<=1	拦截并 toast("■■■■■■..."），结构不崩
误删内容	remove* 删除前 confirm	取消则不删
断签脏值	reconcileStreak	加载时若间隔≥2天归零（按孩子独立）

场景	触发	处理
每日重置	`resetRecurringTasks`	加载/操作时若孩子 `lastDailyReset!=today`，重置其 `daily` 已完成任务为未完成、`lastDailyReset=today`，不扣阳光
删除最后一名孩子	`removeChild` 前置判断 `children.length<=1`	拦截并 `toast("[redacted]")`，结构不崩
重复 task id（导入）	`importFile` 去重	重复 id 追加 `-N` 后缀改为唯一，避免定位冲突
旧版单 profile 导入	`validateImport`	拒绝并提示迁移到多孩子格式
云端无文件（FR-21）	`syncPull` 收到 `404`	`toast("[redacted]")`；不破坏本地
离线/无网络（FR-21）	`syncPush`/`syncPull` 检测到 `typeof fetch ==="undefined"`	抛「离线模式」错误并 `toast`，不崩溃（默认即此态）
同步凭据错误（FR-21）	`doSyncTest` 收到 `401`	`toast("[redacted]")`，不破坏数据
本地与云端冲突（FR-21）	`syncPull` 检测到 `dirty && remoteAt>lastSync`	`confirm` 二选一；取消则抛 `CONFLICT_KEEP_LOCAL` 并保留本地，确定则 `applyRemote` 覆盖本地
阳光值漂移（FR-22）	`load()` 内 `reconcileBalance` 检测到 `ledger` 非空且 `sunshine !== [redacted]`	自动以账本为准修正 `sunshine` 并记 `repair`（不改真实累计）；`ledger` 为空时安全不动、不清零
ledger 过长（FR-22）	`pushLedger` 末尾 / `load()` 内 `pruneLedger` 检测到 `ledger.length > LEDGER_KEEP`	将最早部分汇总为单条 `aggregate` 锚点，运行余额链连续、`Σamount === sunshine` 守恒，存储恒定有界
无 Web Crypto（FR-24）	`HAS_CRYPTO` 为假（旧浏览器/特殊环境）	`hashPin`/`encryptSecret` 退化为 `{v:0, alg:"none"}`，`verifyPin`/`decryptSecret` 走明文兜底，功能不崩（仅失去静态加密，已在文档标注边界）
密文损坏 / 本地密钥丢失（FR-24）	`decryptSecret` 解密失败（`iv`/`ct` 损坏或 KMS 密钥被单独清除）	返回 `null`（区别于「未配置密码」）；`doSyncPush`/`doSyncPull` 检测到 `null` 则 `toast("[redacted]")`；`renderSyncPanel` 将 `null` 归并为 `""` 避免输入框出现字面量 `null`；均不崩溃

8. 安全保密详细设计

• 零网络（默认）：代码默认无任何 `fetch`/`XMLHttpRequest`/外链脚本与字体；离线可用。仅当用户主动启用 FR-21 云同步时，才向其指定的 WebDAV 地址发起 `PUT`/`GET`（及 `PROPFIND` 测试）请求；凭证密码仅在 `rememberPass=true` 时以明文存于本机浏览器、UI 明确警示「仅在你信任的设备勾选」，且上传载荷已剔除 `sync.pass`，不向任何其他服务端发请求。

• 数据本地化：全部存本机 `localStorage`；导出 JSON 含完整数据，`toast` 提示自行保管。

• 权限边界：兑换仅记账，现实活动由家长线下审批；家长审批码按孩子隔离（`cur().pin`）确保兑换动作须家长当面输入当前孩子正确码方可记账，工具无控制外部设备/放行能力（UI 文案明确告知）。

• 防 XSS：所有动态文本经 `escapeHtml` 渲染。

9. 测试设计建议

• 单元：`levelOf` 各阈值边界；`diffDays` 跨月/空值；`computeProgress` 0/部分/全完成；`getRecommendation` 不足 3 项；`toggleTask` 正负对账；`checkIn` 同日/连续/断签三态。

• 集成：load 无数据→建默认并落盘；导入合法/非法文件；导出再导入回环一致。
• 验收（对照 SRS §6）：姓名可改持久化；连续天数正确（含断签/防重）；积分不串；进度条百分比与配色；今日推荐 3 项；导出导入完整恢复。
• 已实现并已通过 Node vm + DOM 桩断言（163 项全过，T1~T39）：等级、断签、同日防重、任务±5对账、ledger 余额一致、进度、推荐≤3、年级切换保留成长记录并载入对应课本、未设码引导设置、设码后正确兑换/错误拒绝、改码须校验当前码、validateImport 好/坏/最小合法、导入覆盖(confirm=true)/坏文件不覆盖/取消不覆盖、打印调用、导出不抛错、重置恢复默认；新增 FR-14：任务增/改名/删+持久化、单元增删、学科增/改名/改色/删、奖励增/改/删（含最少保留）、等级增/改/删+levelOf 用 state.levels、课本编辑跨年级保留（切回恢复）；新增 FR-15/FR-16：多孩子增/切换/删（阳光隔离）、每日周期任务跨天重置且一次性任务不重置、导入规模上限拒绝(总任务>500)、重复 task id 去重后唯一；新增 FR-17/FR-18：T27 连续 7/30/100 天各发一次里程碑奖励（+50/+100/+200）且断签归零后可再获、T28 连续 7 个完美日发放完美周奖励（+60）并中断重置计数、T29 celebrate 在无 DOM 环境安全可调用不抛错；新增 FR-19/FR-20：T30 主题切换与持久化（state.theme 翻转、ensureIds 归一化非法值、非浏览器安全降级）、T31 学科/单元/任务拖拽重排（移动到首位/指定之后、跨层级调用为 no-op、重排后持久化）；新增 FR-21：T32 state.sync 默认关闭且配置字段完整、无 fetch 时 syncPush 安全抛「离线模式」错误不崩溃、上传载荷含 syncedAt 且剔除密码；T33 syncPush 置 lastSync/dirty 并持久化、syncPull 经 validateImport 校验后覆盖本地并保留本机同步配置、404 安全报错；T34 本地与云端均有新改动时 confirm 取消抛 CONFLICT_KEEP_LOCAL 保留本地、确定则 applyRemote 覆盖本地；新增 FR-22：T35 reconcileBalance 在 ledger 非空且 sunshine 与账本派生余额不一致时修正并记 repair、空账本安全不动（不清零真实阳光）、修复后 Σamount === sunshine；T36 pruneLedger 将 >1000 条裁剪为「1000 明细 + 1 条 aggregate 锚点」、Σamount 与 sunshine 守恒、运行余额链连续、重复调用幂等；新增 FR-23：T37 makeDelta 体积远小于整树 clone、未编辑年级返回 null（零存储）；T38 applyDelta round-trip 等价（重命名/改色/新增学科/删除学科/任务 done 状态均保留，未编辑学科与默认种子结构一致）；T39 旧版整树数组格式兼容 clone、且 switchGrade 时自然迁移为差量对象并保留编辑。回归脚本见 verify_dom_stub.js。

---

10. 未决项（与 HLD 一致）

1. 多设备云同步本期已实现为可选 WebDAV/JSON 同步层（FR-21，默认关闭）：沿用离线优先，仅作同步层（参考 starbank WebDAV），不引入云后端强绑定（Cloudflare D1 等按调研归 No-Go）。多孩子（FR-15）已实现，数据模型升级为 children[]。默认课本样例仅作初始化，内容现已支持家长在界面动态维护（FR-14）。
2. 家长审批码（FR-11，按孩子隔离）、打印/A4（FR-12）、导入校验+二次确认（FR-13，v1.4 加固）、内容动态维护（FR-14）、多孩子（FR-15）、周期任务每日重置（FR-16）、庆祝反馈动效（FR-17）、里程碑/完美周（FR-18）、内容编辑器拖拽排序（FR-19）、亮/暗主题切换（FR-20）、可选 WebDAV 云同步（FR-21）、数据层健壮性（FR-22 阳光对账护栏 + ledger 滚动裁剪）、课本差量化存储（FR-23 将 curriculumByGrade 由全量深拷贝改为偏离差量、体积显著下降且旧版整树数组兼容）均已实现并通过 DOM 桩验证（163 项断言：T1~T39）。